

adda: epistemic scaffolding for autonomous data-driven design

Elvis Agüero

Miguel A. Bessa

Draft — September 13, 2020

Abstract

[CONJECTURE] The thesis sentence is deliberately unwritten. It depends on the venue, and `paper/README.md` records three framings the same material can carry — an agents/ML-systems claim about epistemic scaffolding, a computational-mechanics claim about producing correct science, and a systems claim about the tool — each with a different burden of proof. Writing an abstract before that choice would fix the paper’s claim by accident.

What is defensible under all three, and is the honest core: every run closes on a notebook that has been executed and reproduced *before* the close is permitted; claims are carried in a falsifiable ledger whose verdicts are refereed against a written charter at the moment of assertion; everything the agent is told is generated from the live system rather than typed; and the run’s provenance is recorded rather than reconstructed afterwards.

1 Introduction

[CONJECTURE] Venue-dependent, and left as a question rather than guessed at. The three candidate framings and what each must defend are tabulated in `paper/README.md`. This section is written last.

What does not move with the venue is the shape of the problem. An autonomous agent asked to do data-driven design can produce a fluent report of work it did not do, and nothing in a transcript distinguishes that from the real thing. The design decisions in Section 3 are all, in one way or another, answers to that: they make the run’s claims checkable by something other than the agent’s own account of them.

The register in Section 3 is the paper’s centre of gravity. Each entry names a mechanism, what it replaced, and what it costs. Entries whose cost is asserted rather than measured are visible as such, which is the point of writing it this way.

2 Related work

A work is cited here only if it was read in full. Four have been. This is enough to position against and nowhere near a survey; Section 7 records that as the largest known gap rather than disguising it.

Both FEA-agent neighbours route around the documentation problem, and both say so. AbaqusAgent [4] does not ingest the Abaqus documentation at all. It curates 104 worked cases — 71 from the benchmark manual, 33 textbook-style — embedded into a single vector store, with a whole case as the retrieval unit and an explicit prompt instruction that a retrieved case is a template for structure, not physics. The authors give the reason plainly: the documentation of a closed-source commercial package is not readily available, and curating it is a significant challenge under limited

resources. PDE-Agents [6] likewise holds material properties, failure patterns and run lineage in its knowledge graph rather than manuals.

The measured benefit of retrieval in this neighbourhood is efficiency, not accuracy. AbacusAgent’s ablation (20 cases, one model, temperature 0) reports 90% success with retrieval against 80% without, and 634,690 tokens against 2,203,778 — $3.5\times$ fewer tokens and $2.2\times$ less time for +10pp, an accuracy delta they do not claim is significant at that n . This directly shapes our endpoint choice (Section 5). Two cautions carry over. Their evaluation set of 50 draws 40 cases from the 104-case retrieval library, which inflates any retrieval-on condition. And the same table run on a different model moves from 90%/90% to 65%/45% at fixed architecture: **model choice moved the result more than the ablated feature did**, which is why every arm here pins and names its model.

Integration pattern beats index choice — two independent groups. PDE-Agents states it outright: integration pattern, rather than knowledge content, decides whether graph-augmented retrieval helps or hinders. [2] finds grep beating vector retrieval in its first experiment, but its own conclusion is that the harness and tool-calling style dominate the retrieval strategy. If that holds, ablating *how* a capability is wired into the loop is more informative than ablating *which* retriever is used — which is the shape our evaluation already has.

What does not transfer, recorded because it was nearly over-read. [2] benchmarks on LongMemEval, conversational memory where answers are often licensed by a small set of distinctive strings: the best possible case for lexical matching and a poor analogy for technical documentation. [1] reports no measurable benefit from chunk overlap, for a configuration (open-domain Wikipedia QA, a sparse-learned retriever, a small generator) that shares none of our parameters. Both findings are real and neither bears on our setting.

The opening, stated carefully. Nobody in this neighbourhood has made a full documentation corpus traversable; both neighbours hand-curate a small structured knowledge base instead. Two risks before claiming that gap. It is adjacent to graph-augmented retrieval, which PDE-Agents has already applied in this exact domain, so the difference must be principled — a hierarchy *derived* from document structure versus a *curated* graph of facts. And given the integration-beats-index finding, a traversable tree without a traversal policy would likely underperform its own promise.

3 Method

This section is a *register*, not a narrative. One entry per mechanism, each owing the reader the same three things: what it does, what it is instead of, and what it costs. The shape is enforced by a macro, so an entry that omits its cost is a visible hole rather than a stylistic choice.

It is written to grow past any page limit on purpose and is cut at submission. A design decision taken and not recorded here is lost; a decision recorded and later cut costs nothing.

Write each entry as non-technically as its subject allows — see rule 0 in `paper/README.md`. Say what a mechanism DOES rather than what it is called; name an identifier only where the name is itself the decision or a reader would need it to find the thing. This section is where the rule slips first, because every entry is written just after the code it describes and arrives carrying that code’s vocabulary. The reader is a researcher deciding whether to believe the design, not somebody who will maintain it.

3.1 The graph and the open loop

Hub-and-spoke topology A strategizer holds outgoing edges to a literature reviewer, a data generator, an implementer and a critic. Behaviour follows topology rather than role: any node with an outgoing edge orchestrates, whatever it is called, so a two- or three-node study runs without a separate code path. *Instead of:* a fixed pipeline of stages, or a fully connected team. The hub keeps one agent accountable for the run’s scientific direction while letting specialists be swapped or removed. *Costs:* the hub is a bottleneck and carries the longest context. Removing a specialist is a graph edit — which is what makes the evaluation’s leave-one-out arms cheap.

Delegation is asynchronous, and harvesting is one call Work is dispatched without blocking, and a bare `Wait()` returns whichever delegation finishes first, so N in flight are drained by N calls. It refuses rather than hanging when every open delegation is parked on a question or has died without reporting. *Instead of:* a single-target wait, which left polling as the only way to collect a fan-out — and polling is discouraged elsewhere in the prompt, so the two rules quietly opposed each other. *Costs:* concurrency is a behaviour, not a guarantee. Across 39 cluster runs, reliance on the single-target wait predicted serial execution ($r = -0.54$ against mean concurrent delegations, controlling for duration) at a measured mean concurrency of 1.21 on a median of 15 delegations per run.

3.2 The epistemic layer

Four mechanisms, deliberately separable so each can be ablated alone.

Hypothesis ledger An append-only record of falsifiable hypotheses and their verdicts (OPEN, SUPPORTED, FALSIFIED, INCONCLUSIVE), mutated only through tools and never edited directly. Every delegation cites the hypothesis it tests. *Instead of:* letting the agent keep its claims in its context window. The ledger survives context compaction, is auditable afterwards, and makes “what did this run actually claim” a file rather than a reading exercise. *Costs:* prompt surface and tool calls; and because delegations must cite a hypothesis, it shapes how work is decomposed — a benefit or a constraint depending on the problem.

The record is strict, and the strictness is the mechanism: a claim with no falsification criterion cannot be written down at all.

```
{ "id": "H1",  
  "statement": "Optimal t/L is near 0.08 - thin walls maximise buckling",  
  "falsification_criterion": "a point with t/L in [0.10,0.14] beats  
                           buckling_load_norm 1.47",  
  "prediction": "best at t/L ~ 0.08 +/- 0.02",  
  "prior": 0.55, "posterior": 0.78,  
  "status_log": [{"status": "OPEN",      "delegation": null},  
                 {"status": "SUPPORTED", "delegation": "D006",  
                  "numbers": {"buckling_load_norm": 1.62}}]}
```

Falsification charter One binding text defining how a hypothesis may be tested and labelled — severity of the attempt, verdict follows the result, no goalpost-moving — spliced into both the strategizer’s and the critic’s prompts so both cite the same standard. *Instead of:* per-agent instructions that drift apart. A shared constant is the cheapest way to make two agents answerable

to one rule. *Costs*: a fixed block of prompt in two roles, and a rule the agent can be held to only as well as the referee below enforces it.

Live verdict referee When a hypothesis is closed, an independent judge checks *at the moment of assertion* that the verdict obeys the charter, and is fed its own prior rulings on the same hypothesis so a borderline verdict cannot silently flip. It runs on the critic’s model, not the strategizer’s, and is advisory: on timeout or failure the verdict stands. *Instead of*: checking only at the end, when the run has already been built on the claim. *Costs*: an extra model call per verdict, budgeted tight — a hung call once froze a run for roughly 89 minutes, which is why the timeout exists. Activation 8.7% of runs¹, which makes it descriptive rather than inferential in any ablation.

Drift monitor Background rules bracketing the evaluation ledger from both directions — evaluations reported but never written, and rows written with no attributable owner — escalating repeated drift to the critic. *Instead of*: trusting the agent’s account of what it evaluated. *Costs*: activation 19.6%²; also descriptive rather than inferential.

Process milestones A small backlog — assess the literature, get the oracle ready — that gates delegation to the implementer until the strategizer resolves each one, by completing it or skipping it with a reason. *Instead of*: trusting that an agent will orient itself before spending real evaluations. The gate is on the ONE node that consumes budget, so exploration stays free. *Costs*: it fires in 93.5% of runs³, so it is the highest-dose mechanism here and the one most likely to be shaping behaviour in ways not attributed to it. Skipping is soft, which means a determined agent can clear the backlog without doing the work.

Pre-registration reaches the experimenter The registered prediction and its falsification criterion are injected into the worker’s brief, so the agent running the experiment can read the claim it is testing. *Instead of*: registering a prediction where only the strategizer can see it. A criterion the experimenter never reads cannot discipline the experiment; it can only be checked against afterwards, when the work is already done. *Costs*: it couples two prompts, and it lets a worker optimise toward the criterion rather than toward the truth — which is the same hazard pre-registration carries in human science, and is why the referee above judges the verdict rather than the result.

3.3 The deliverable and the reproduction gate

Reproduce before close The run closes on a notebook that has been executed and reproduced *before* the close is allowed. A deliverable that fails to reproduce after a bounded number of sighted attempts closes the run FAILED — loudly, and distinctly from a conclusion that was merely never validated. *Instead of*: a report plus a promise. This converts “the agent says it found X ” into “here is a notebook that re-derives X from the recorded data”, and is the mechanism most likely to be the headline for a mechanics audience. *Costs*: it constrains what the agent may produce, and the gate is a real failure mode: reproduction bounces in 13.0% of runs⁴.

¹Computed from `studies/run_ledger.csv`.

²Computed from `studies/run_ledger.csv`.

³Computed from `studies/run_ledger.csv`.

⁴Computed from `studies/run_ledger.csv`.

3.4 Ground truth and the ledger

One metered door to the oracle Every ground-truth evaluation goes through `get_evaluator()`, which wraps the registered generator so that each call is written to the canonical `ExperimentData` store with its provenance, under a file lock. The wrapper is not exported: an agent cannot construct a store-redirectioned evaluator. *Instead of:* letting an agent call the objective function and report what it found. The ledger is then the run’s evidence, and a claim is checkable against rows rather than against the agent’s account of them — which is what makes the reproduction gate above possible at all. *Costs:* one door is a bottleneck by design, and everything cheap that is NOT an evaluation (surrogate predictions, analysis) has to be explicitly off-ledger or the meter stops meaning anything.

Design namespaces: several oracles in one run A run may carry more than one oracle, one per design parametrization the agent invents, each with its own isolated store. `Delegate(..., namespace=...)` scopes a worker to one, injected through the environment so the agent’s call site stays a bare `get_evaluator()`. *Instead of:* one fixed design space per run. Open design-space discovery means the agent proposes a parametrization the study did not anticipate, and it needs somewhere to evaluate it that cannot contaminate the baseline. *Costs:* additive by construction — `namespace=None` is byte-for-byte the single-study path — but it means “the ledger” is N ledgers, and the single-store load idiom silently reads only the default. That is a live trap, which is why the deliverable spec names the multi-store call.

Counts are derived from the ledger, not remembered A report-time tool reads per-experiment and per-delegation evaluation counts live from the stores, and prints budget spent against remaining. *Instead of:* letting the agent quote a number from its plan. Both failure modes are observed, not hypothetical: one run hardcoded 70 evaluations where the ledger held 90, and another asserted 200 remaining with 200 spent. Both closed UNGATED. *Costs:* another tool in the catalogue, and a discipline the prompt has to insist on — the agent must call it rather than recall.

3.5 Anti-drift: generated, not typed

The prompt is derived from the live system The tool catalogue is rendered from the live closure dictionary at every invocation; the run diagram is generated from the live graph; a feature owns its knob, its tools and its prompt section as one unit; and the documented run-knob table is checked against the source, so a knob read but undocumented, documented but unread, or documented with the wrong default fails the build. *Instead of:* documentation and prompts maintained in parallel with the code. The failure this prevents is specific and was observed repeatedly: a flag that appears to disable a feature while the agent is still instructed to use it. *Costs:* generation makes the assembled prompt harder to read as one artifact, which is why a provenance map exists citing every prompt fragment back to its source line. And the guarantee is only as good as its contract tests: one such test was silently reduced to checking one of seven feature defaults by a refactor that moved defaults into the feature registry, and reported as passing skips throughout.

3.6 Provenance: recorded, not reconstructed

Terminal state is a value, not a grep The run records rather than re-derives its outcome, *how* it terminated, *whether anything reviewed it*, the resolved configuration it actually ran with after precedence, and per-call telemetry by role, phase and model. *Instead of:* inferring these afterwards from artifacts. *Costs:* a schema to maintain, and a migration for historical rows which cannot be back-filled honestly and are left blank. The argument for paying it is empirical and is the strongest

single piece of evidence here for the whole provenance stance: when the outcome *was* inferred — grepped from the run’s own final report — backstop kills and critic-less runs both recorded as validated successes, and those rows are still in the ledger.

One git repository per run workspace The run’s delegation workspace is a git repository of its own, with one commit per delegation — succeeded and failed alike — and the resulting sha stamped onto that delegation’s record. *Instead of:* taking a delegation’s word for which files it changed. The reproduction gate proves the notebook runs; it cannot prove a worker’s account of its own edits is faithful. *Costs:* the agents get no git tool and never see the repository: the harness commits on their behalf, because an agent that can rewrite the history recording its work defeats the purpose. Never fatal — no git, or a failed commit, records a null sha and the run proceeds.

3.7 Watching and steering a run in flight

A read-only live viewer A local web UI serves the agent graph with a live status per node and, on click, that node’s running transcript. Read-only, and a separate process from the run. *Instead of:* reading a log after the fact. A multi-hour autonomous run that has gone wrong in the first ten minutes is the expensive failure, and nothing in a transcript tells you that while it is still cheap to stop. *Costs:* it is a second consumer of the run’s on-disk state, so anything it displays is a format the run is now committed to writing.

The operator channel is the file system A human can answer a worker’s question, queue a note for the entry node, or address a note at a specific running delegation — all as small JSON files in the run’s own debug directory. *Instead of:* a socket or an RPC. The run and the viewer are separate processes, so the file system is the simplest channel that works — and it makes the whole exchange part of the run record rather than terminal scrollback. *Costs:* it introduces a watch heartbeat, because a run must be able to tell waiting-for-an-answer apart from stalling on a question nobody can see.

3.8 Resource governance

One hard boundary, soft budgets everywhere else A memory cap for host safety is hard. Everything else is soft: the evaluation budget nudges rather than stops, and wall-clock and cost act as resumable backstops rather than crashes. *Instead of:* hard limits on the science. A budget that kills a run mid-conclusion destroys the run’s value, so backstops checkpoint and halt resumably. *Costs:* a resumable halt is a state the analysis must handle, and it is the source of the censoring problem in Section 5.

A wall-clock watchdog outside the run A hard timer at twice the run’s time budget force-exits a stalled run and recursively kills every process tree it spawned, including detached ones a process-group kill would miss. It leaves a labelled post-mortem. *Instead of:* trusting the run’s own backstops. Those are inside the process that may be the thing that is stuck. *Costs:* it is an EXTERNAL cutoff, not an error condition, and it is the source of the censoring problem in Section 5: killed runs carry no token or cost data, and at least one had already reached a passing critic verdict. It is never pooled with failure.

Compaction rather than trimming, when the window fills When a conversation outgrows the served context window, the middle is replaced by a generated summary that keeps numerical results and the delegation each came from, decisions and their reasons, and what was tried and failed. The first user turn is never evicted. *Instead of:* dropping the oldest messages, which is free and deterministic and also blind: the message it evicts may be the delegation report the whole run turns on, and nothing downstream can tell that it went. *Costs:* a summary is model output, so it can be wrong in ways a dropped message cannot — it can quietly restate a number. So it is fenced and labelled in the transcript, the full transcript still lands on disk, and the run records which policy it used so an analysis can condition on it. A failure to summarise falls back to trimming rather than failing the turn.

3.9 Knowledge the agent cannot hold in context

Five bodies of reference material are indexed rather than injected. The design of each is dictated by the form the material takes, and the differences between them are the interesting part.

f3dasm: runtime introspection The installed f3dasm is imported and introspected at run time — 243 symbols, roughly 117,000 characters of docstring — and exposed as a single lookup tool to the two agents that write f3dasm code. Each entry leads with the *canonical public import path*, resolved by walking the public namespaces. *Instead of:* parsing a source checkout. On a cluster the checkout may be a different version or absent, so an index built from the live import describes the exact code the run will execute against and cannot go stale. The public-path resolution is not a refinement but the point: `__module__` reports the private definition site for every public f3dasm symbol, and an index that handed that to an agent would produce fragile private imports — worse than no tool. *Costs:* importing the package, with its side effects. Only 47 of 125 defined symbols are publicly reachable; the other 78 are answerable but marked “do not import this”, so a lookup cannot be mistaken for an endorsement.

Abaqus: offline extraction, BM25 over structure Licensed documentation HTML is parsed once into a page database and searched with BM25 [7] over weighted title, abstract and body fields. The builder ships; the corpus does not. *Instead of:* flattening pages to text. A survey of 1,200 pages found 99% carry tables, 52% MathML and 37% preformatted blocks, with parameter documentation in definition lists — exactly the content a reference lookup needs, and exactly what flattening destroys. BM25 rather than the f3dasm ranker because this corpus is large and wildly non-uniform: document-length normalisation and inverse document frequency both matter here and neither is available from a few hundred uniform symbol entries. *Costs:* the corpus cannot be redistributed, so the feature is opt-in and unmeasured until a user supplies one. Exact-token search also needs a spelling bridge, since every Abaqus keyword begins with a character the tokeniser discards.

adda: the same introspection, a different unit mix adda indexes itself for *other people's* agents, exposed as `adda-docs`. Four unit kinds are indexed: symbols, module docstrings, declared module-level constants (each carrying its module's docstring), and the knowledge-base markdown shipped inside the package. *Instead of:* indexing symbols alone, which is what f3dasm needs. Measured on 128 labelled queries, symbols alone *lose* to `ripgrep`; all four together beat it. The same four units measured on f3dasm make it worse. The unit mix is a property of where a package keeps its knowledge, not a global setting: 23% of adda's docstring mass is in module docstrings, and its rules live in markdown. *Costs:* a fourth index to maintain, and a known weakness — there is

no minimum-score floor, so the tool almost never reports that it does not know, and a confidently wrong answer costs an agent more than a miss.

Basilisk: the source tree, at the granularity it is authored in The Basilisk CFD source is indexed as solver headers and worked cases, with each header’s entry naming what it is VERIFIED WITH and what it is NEVER STACKED WITH — derived from what 213 compiling cases actually include together. *Instead of:* an abstract-syntax index. Measured on the real tree: a C grammar encodes operator precedence as depth, so 59k lines becomes roughly two million nodes at depth 100 with three-quarters carrying no information; Basilisk C is a DSL, so a C parser emits error nodes on exactly the constructs carrying the physics and discards as comments the literate prose that is 16% of every solver file; and the headers do not parse standalone even with Basilisk’s own parser. *Costs:* the compatibility signal is an ABSENCE, and absences lie. Raw co-occurrence called two headers incompatible when one reaches the other transitively, which would invent a constraint the physics does not have — worse than a miss. The fix excludes each header’s transitive closure and ignores any header too thinly used for its absence to mean anything.

Literature: a study-scoped corpus of primary sources Papers the literature reviewer acquires are chunked, embedded and searched, and the corpus persists across runs of a study. *Instead of:* re-acquiring and re-embedding per run, which is waste. *Costs:* it is a contamination channel: a run can answer using papers an earlier run fetched, so any arm that ablates the literature stack needs a cold corpus or an explicit note. Recorded in Section 6 as accepted rather than fixed.

One contract across all five, and deliberately not one mechanism Every provider answers `consult(query, limit)`: a key returns its entry, a phrase returns a menu, a miss says the miss is about THIS index rather than about reality, the reply is capped, and an unconfigured provider returns no tool at all rather than one that errors. *Instead of:* five providers converging by imitation, which is what had happened. Imitation does not fail when somebody drifts, and two providers had shipped with no reply cap at all — defeating the one thing the tools exist for. *Costs:* the MECHANISMS stay different, on evidence: f3dasm gets worse with adda’s unit mix (r@1 0.77 → 0.62) and adda loses to `ripgrep` without it, while Abaqus needs length normalisation that f3dasm’s uniform entries neither have nor need. Forcing one ranker would make three providers worse to make a table look tidy.

3.10 What this section still owes

1. **The alternatives in the author’s voice.** Most “instead of” lines are reconstructed from code comments. The real abandoned alternatives are in commit messages and the backlog, and are more convincing than a retrospective rationalisation.
2. **A worked example.** One run end to end with its actual artifacts: problem statement in, hypotheses proposed, a delegation, a verdict and its referee ruling, the gate firing, the notebook out.
3. **Quantified costs.** Several costs asserted qualitatively above are measurable from telemetry and should be numbers.

4 Implementation

Short by design. The graph runtime is LangGraph [3]; the data-driven primitives are f3dasm [8]; the agent surface is declared per role. The one part worth a reader’s attention is Section 3.5, and it belongs there rather than here.

5 Evaluation

Fixed before any run, so the analysis cannot be chosen after seeing the data. Every figure below is computed from `studies/run_ledger.csv` and is reproducible from it.

5.1 What the existing data is, and is not

The ledger holds 76 runs across 66 distinct commits⁵ — close to one commit per run. It is a longitudinal log, not an experiment: every row confounds a feature change with everything else in that diff. It is used *only* to estimate variance and activation rates, never as a result. The anchor study ($n = 46$) closes GATED 28 times, UNGATED 6, FAILED 2, and is watchdog-killed 10 times.

5.2 Two findings that constrain the design

Dose: most features do nothing in most runs. Activation, as the fraction of the 46 anchor runs in which a signal fired at least once: milestone block 93.5%, error return 78.3%, science drift 19.6%, consistency flag 15.2%, reproduction bounce 13.0%, verdict-substance flag 8.7%. A feature that does nothing in 80% of runs has an intent-to-treat effect near 1.0 whatever it does to the runs it touches: a true $1.5\times$ effect among treated runs becomes roughly $1.08\times$ in ITT, and the required n rises by more than an order of magnitude at the same variance. **Consequence, pre-registered:** activation rate is a manipulation check reported per arm, and features below roughly 20% dose are reported as descriptive, not inferential.

Censoring: the endpoint is missing exactly where it matters. All 10 watchdog-killed runs carry no token and no cost data, and the watchdog is an *external* wall-clock cutoff rather than an error — at least one killed run had already reached a critic PASS. The continuous endpoint is therefore computed over survivors only, and survival is post-treatment: if disabling a feature makes runs hang, those runs vanish from the token analysis and the survivors look cheaper, so an arm could be reported as a cost saving when it in fact broke the system. **Pre-registered:** killed runs are reported as right-censoring and never pooled with failures; completion rate is reported per arm beside the endpoint; the token result is stated as conditional on completion; and attrition of 21.7% means retaining 10 usable runs per arm needs ≈ 13 launched.

5.3 Endpoints

Primary, continuous: tokens to outcome. Over the 36 completed anchor runs the log-scale spread is small — $\text{sd}(\log) = 0.321$ for output tokens, giving $n = 10$ per arm for a $1.5\times$ effect. Cost gives 0.354 and delegations 0.438. Ledger rows are excluded as an endpoint: at $\text{sd}(\log) = 1.064$ they are noise at this scale.

Secondary, binary: success rate, with a Wilson interval [9] and an explicit underpowered label. Detecting a shift from the observed 0.61 to 0.71 at $\alpha = 0.05$ and 80% power needs ≈ 351 runs

⁵Computed from `studies/run_ledger.csv`.

per arm, which is why the binary endpoint cannot be primary on a paid model. On an open-weight backend cost is unavailable entirely, so tokens and wall-clock carry the analysis there.

5.4 Design

Leave-one-out against a full baseline; twelve factors full-factorial is 4096 cells and is not on the table. A pair is promoted to a 2^k design only where one-factor-at-a-time suggests coupling. The model is pinned and named — [4] moves from 90%/90% to 65%/45% purely by changing model at fixed architecture. Arm order is randomised across the sweep so condition is not confounded with time, replicate is a blocking factor, and analysis is on the log scale.

Two factors are settled. The **delegation log is not ablatable**: it is run substrate. The **critic arm cannot use the run outcome**, because with no critic nothing reviews the conclusion — it needs a post-hoc judge over the archived deliverable, blinded to condition. The **hypothesis-ledger arm is partial** and must be reported as such: its tools and prompt section can be withheld, but the Popperian workflow is argued across three other prompt sections that are the agent’s scientific method rather than ledger documentation, and removing those would produce a different agent, not an ablation.

5.5 Retrieval, measured separately

The knowledge tools of Section 3.9 are measured offline against labelled query sets, because a run-level arm says whether an arm moved and not why. The honest control is **ripgrep** on the same questions, not nothing, and the paired test is exact McNemar [5].

Query sets are written before the tool, and by someone else. This is not a stylistic preference. f3dasm’s ranker carries a hand-written English-to-f3dasm alias map added after observing which queries missed. On the 56 queries it was written from it lifts the synonym tier from $r@1 = 0.29$ to 0.47. On 70 queries written by an agent barred from running the retriever it scores 0.06, against 0.12 with the map disabled, and only one of its 84 keys is triggered by both sets. It memorised a vocabulary rather than learning one, and no measurement taken on the set it was tuned against could have shown that.

Result for the adda index, over 128 queries (48 written alongside the tool, 80 written blind): $r@1 = 0.375$ against 0.23 for **ripgrep**, $MRR = 0.459$ against 0.32, exact McNemar $p = 0.0096$. The shrinkage between the two sets is itself the finding — $r@1$ falls from 0.40 on the set written alongside the tool to 0.31 on the blind set, the same effect the alias map shows at full strength.

5.6 Unanswered before the first run

What is the baseline? Leave-one-out gives within-system comparisons and does not answer “compared to what” — an agent with no scaffolding, a competent human, or the same system on an easier problem. This must be named before the runs. **Is the task set off the floor?** If baseline success is near zero no ablation can move it; the fix would be an easier task tier, not a larger n . **A pilot is required.** Every figure here was measured on one model and one study. Roughly 10 baseline runs, measured, before any arm is sized.

6 Threats to validity

Several of these are the most defensible content the paper has. They were found by the system reporting something untrue and being caught, which argues for the provenance machinery better

than any claim we could make for it.

6.1 Measurement — things the system got wrong about itself

The run’s outcome was recovered by grepping its own prose. Terminal state was determined by searching the final report for banner strings and defaulting to success when none matched. Two classes of non-success were therefore recorded as validated successes: a run killed by the time backstop, the cost ceiling or repeated errors, whose banner matches no pattern; and any run in a graph with no critic, where no banner is emitted at all. Rows of both kinds are in the ledger now, so any analysis of historical data must treat the success label as unreliable before the fix.

A feature’s flag did not turn the feature off. Tools were gated on a static declaration rather than on whether the backing object existed, and each role prompt was one literal string. A disabled feature left its tools registered and its instructions in the prompt; the agent called a tool, received an error string, and the run recorded the one diagnostic whose target is zero. Three successive backlog entries are three rounds of this for a single feature. An arm built that way measures an agent confused by broken tools.

Cost was unmeasured for an entire class of run. Tokens and cost were parsed out of the stamped deliverable, so a study configured without a notebook recorded no cost at all, and only two of four token fields survived — the two dropped being the cache fields, which are precisely what a prompt-section change moves. Unpriced calls were summed as zero, so an open-weight run reported a cost indistinguishable from free.

A contract test disabled itself. The guard asserting that every documented knob default matches the source scrapes call sites for a literal. Moving feature defaults into the feature registry left no literal at the call site, so six of seven feature knobs became *passing skips*. Nothing was wrong underneath, and nothing would have said so if it had been.

Implication. Any figure computed from a run predating these fixes inherits them. Section 8 states which figures fall on which side of that line, and the ablation runs on the fixed system.

6.2 Statistical

Informative censoring and **low dose** are stated with their pre-registered handling in Section 5. **The variance prior does not transfer:** every dispersion figure was measured on one study, one model, and runs spanning 66 commits. **No correction for multiplicity** is applied: a leave-one-out sweep over twelve factors is twelve comparisons, and we either pre-register the primary factor or report adjusted intervals — not both, post hoc.

6.3 Construct

“Success” is the system’s own gate. It means a critic agent of the same model family passed the conclusion; that is not ground truth, and for the critic arm it is not even meaningful. **Partial ablations** answer narrower questions than their names suggest and are reported as the narrower ones. **Judge and judged share a model** — the critic, the verdict referee and the agent under review are one family, so shared blind spots are invisible to this design.

6.4 External validity

One system, one framework: no claim transfers without re-measurement. **Small task set,** and the anchor study is not committed to the repository — both an external-validity threat and a reproducibility defect. **Model dependence is large and demonstrated** [4], so any single-model result is a statement about that model.

6.5 Accepted deliberately

Cross-run corpus reuse: the literature corpus is study-scoped by design, so a run can answer using papers acquired by an earlier one. This is intentional — re-embedding every run is waste — but it is a contamination channel for literature arms, which need either a cold corpus or an explicit note. **Study-scoped deliverable:** parallel runs of one study are unsafe without a per-run copy; the sweep layer owns this. **No seed or temperature control** exists anywhere, so replicates are the only handle on run-to-run variance and there is no “same seed, one flag flipped” comparison available. The paper must not imply otherwise.

7 Limitations and future work

Related work is four papers. Enough to position against, nowhere near a survey. Missing entirely: autonomous-scientist systems outside FEA, the self-verification and self-consistency literature, agent memory and provenance, and evaluation methodology for agentic systems — which is the literature this paper’s evaluation design should itself be answerable to.

No results. The evaluation is designed, pre-registered and unrun.

The retrieval tools are unevenly measured. adda’s index has 128 labelled queries and a baseline; f3dasm’s has 126 and no baseline comparison against grep; the Abaqus tool has 60 queries and 50 fuzzy pairs written but *cannot be scored*, because the corpus is licensed and not shipped. The asymmetry is stated rather than hidden.

Deliberately not built. Full run-scoped isolation; a minimum-score floor on the knowledge tools, which cannot be judged by recall (a floor can only lower it) and needs a set of questions the system genuinely cannot answer, scored on how often it says so; and the traversable documentation tree of Section 2, which needs a traversal policy before it is worth building.

8 Reproducibility statement

Written early on purpose: it disciplines the rest.

Which figures are recomputable, and from what. Activation rates, outcome counts and all dispersion figures come from `studies/run_ledger.csv`. Retrieval figures come from the labelled query sets in `tests/` and are reproduced by `internal/tools/source_index_baseline.py`. Prompt provenance is regenerated by `make promptmap` from the live graph objects, and a test fails if a prompt moves out from under its citation.

The line historical data falls on. Figures computed from runs predating the fixes in Section 6 inherit those defects; the ledger’s success label is unreliable before them. The ablation runs on the fixed system, and the two sets of figures are never mixed in one table.

Known reproducibility defects. The anchor study that every variance figure is drawn from is not committed to the repository. No seed or temperature control exists. The Abaqus corpus cannot be redistributed, so that tool’s quality is unmeasured by anyone without a licensed installation.

What is pinned. The model is named per arm. The resolved configuration each run actually used is recorded per run rather than inferred from the study’s configuration file.

References

- [1] Authors of arXiv:2601.14123. A systematic analysis of chunking strategies for reliable question answering. 2026. arXiv:2601.14123. READ IN FULL. Reports no measurable benefit from chunk overlap FOR ITS CONFIGURATION (Natural Questions, SPLADE, Mistral-8B). Recorded because it was initially over-read as a general result. [VERIFY] author list against the source.
- [2] Authors of arXiv:2605.15184. Is grep all you need? how agent harnesses reshape agentic search. 2026. arXiv:2605.15184. READ IN FULL. Grep beat vector retrieval in their Experiment 1, but their own conclusion is that the harness, not the retrieval strategy, dominates. Benchmark is LongMemEval, where answers are licensed by distinctive strings — the best case for lexical matching and a poor analogy for technical documentation. [VERIFY] author list against the source.
- [3] LangChain, Inc. Langgraph. <https://github.com/langchain-ai/langgraph>. The graph runtime. Cited as an implementation detail, not a contribution.
- [4] LIRAM-LIN authors. Abaqusagent: a multi-agent framework for end-to-end finite element analysis. 2026. arXiv:2606.00138. READ IN FULL. Nearest neighbour. Does not ingest documentation: curates 104 worked cases in one FAISS store, retrieval unit is a whole case. Ablation (n=20, Opus-4.6): RAG on 90% success / 634,690 tokens vs RAG off 80% / 2,203,778 — the large effect is efficiency, not accuracy. Same table on GPT-5.2 gives 65%/45%: model choice moved the result more than the ablated feature. 40 of 50 eval cases are drawn from the retrieval library. [VERIFY] author list and exact title against the source.
- [5] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. The paired test used for every index-vs-baseline comparison here. The systems answer the same queries, so an unpaired two-proportion test is the wrong instrument and less powerful.
- [6] PDE-Agents authors. Pde-agents: Llm-orchestrated multi-agent finite element modelling with knowledge-graph reasoning. 2026. arXiv:2606.07850. READ IN FULL. Also routes around documentation: the graph holds material properties and run lineage, not manuals. Headline we lean on: “integration pattern, rather than knowledge content, [decides] whether GraphRAG augmentation helps or hinders”. [VERIFY] author list and exact title against the source.
- [7] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. BM25, used for the Abaqus corpus. Cited for the two properties that decide where it belongs: IDF and document-length normalisation, neither of which the f3dasm ranker has or needs.
- [8] Martin van der Schelling and Miguel A. Bessa. f3dasm: Framework for data-driven design and analysis of structures and materials. <https://github.com/bessagroup/f3dasm>. The substrate adda orchestrates. [VERIFY] — there is a published paper for f3dasm; cite that rather than the repository once its details are confirmed.

- [9] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. Interval used for every rate reported. At $n=34$ with p near 0.3 the normal approximation runs past 0 and 1.